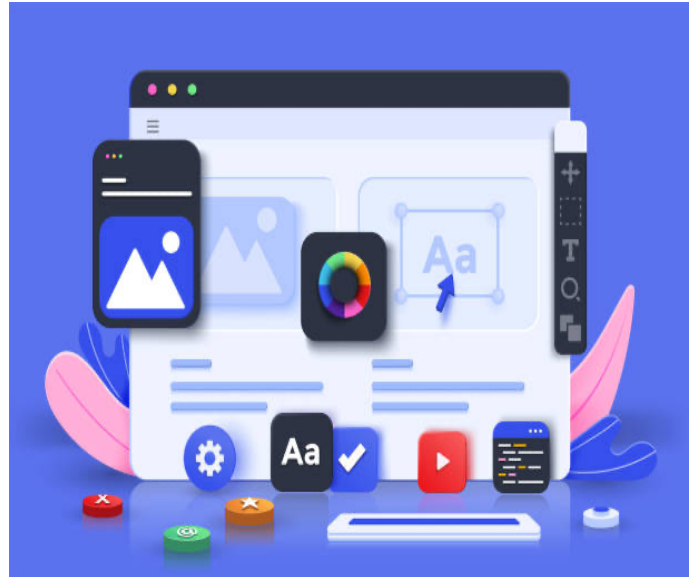


SD

—



Module Mijn eerste webgame

Instructie 06 – Klikken op een cel programmeren

- Voorbereiden
- Pseudo-code voor een complexe IF-statement
- Helperfunction isDraw

Auteur:	Johan Strootman
Afkorting:	SMN
Versie:	juli 2023

Inhoudsopgave

Inhoudsopgave	3
INLEIDING	6
Waarom is dit de hoofdfunctie?	6
Welke vragen roept het programmeren van deze functie nu al bij ons op?	6
WAT IS DE CELLCLICK FUNCTIE VOOR EEN FUNCTIE?	8
EventHandler	8
Wat is een parameter?	9
Voorbeelden	9
Voorbeeld 1	9
Voorbeeld 2	9
Parameter van een EventHandler	10
Voorbeelden	10
Voorbeeld 1	10
Voorbeeld 2	10
Parameter = Argument	11
WELKE GEGEVENS VINDEN WE IN DE PARAMETER VAN CELLCLICK?	12
Testen	13
Welke property in event_element kunnen we gebruiken?	14
Conclusie	14
SRC-ATTRIBUUT IMG-TAG VERGELIJKEN MET TEKST	15
Inleiding	15
Wat betekent dit voor ons?	16
ZOEKEN NAAR EEN ANTWOORD OP ONZE VRAAG	17
Inleiding	17

Wat is een betere zoekvraag voor het internet?	17
Wat vinden we dan?	17
CELLCLICK FUNCTIE NU AFMAKEN	19
Wat moet de cellClick functie nu eigenlijk doen?	19
De taken	20
Taak: Speler actie uitvoeren.....	21
Resultaat in code.....	21
Toelichting op de code	22
Win situaties controleren	23
Inleiding	23
Van pseudo-code naar JavaScript.....	23
Eerste pseudo-code controle omzetten naar JavaScript	24
Opmerkingen vooraf	24
Resultaat.....	24
Vertaling van de complete pseudo-code	25
Pseudo-code (compleet voor 1 speler)	25
Eindresultaat.....	27
Programmeren van de acties.....	28
Speler 1	28
Toelichting op de code	29
Speler 2	30
Toelichting op de code	30
Gelijkspel	31
Hoe pakken we het dan nu aan?	31
Hoe gaat de functie dan heten?.....	31
Zo zou de functie er dan uit kunnen zien	32
.....	32
Toelichting	32

Hoe past dit in de gehele controle?	33
---	----

INLEIDING

We gaan nu beginnen met het programmeren van, in feite, de hoofdfunctie van ons spel.

Waarom is dit de hoofdfunctie?

Het is de hoofdfunctie, omdat iedere ronde die gespeeld wordt door deze functie moet worden afgehandeld. Deze functie controleert b.v. of een cel leeg is en gevuld kan worden door een speler, of na iedere zet van een speler er een win-situatie is ontstaan, of er na iedere zet een gelijk spel situatie is ontstaan enz.

Welke vragen roept het programmeren van deze functie nu al bij ons op?

De volgende vragen worden bij ons opgeroepen nog voor we beginnen aan het programmeren:

We weten dat alle cellen (oftewel alle IMG-tags van het speelveld) verzameld zijn in 1 variabele, namelijk **element_cells** en we weten dat er **slechts één functie** is die moet reageren op een klik op welke cel dan ook, want zo hebben we dat geprogrammeerd in de **cellClick** functie:

```
/*  
    In de onderstaande programma lus lopen we nu langs alle cellen in het speelveld  
    en koppelen hier een click eventhandler aan.  
*/  
element_cells.forEach(cell => {  
    cell.addEventListener('click', cellClick);  
});
```

Figuur 1 - cellClick functie

Namelijk de cellClick functie (zie hierboven).

Maar de vraag die we ons zelf nu moeten stellen is:

'Hoe weet de cellClick functie dan op welke cel er geklikt is?'

Want wat deze functie moet doen is afhankelijk van op welke cel geklikt is.

Dit moeten we dus eerst onderzoeken en ontdekken. Want zonder te weten hoe kunnen we niet eens beginnen aan het programmeren van deze functie.

Iedere cel in het speelveld is in feite een **IMG-tag**, dus een *afbeeldingsbestand*. Wanneer een cel leeg is wordt eigenlijk het *afbeeldingsbestand* **img/empty.jpg** getoond. Wanneer een speler zijn/haar symbool in een cel heeft kunnen plaatsen kan één van de volgende twee *afbeeldingsbestanden* getoond worden: **img/cross200x200.png** of **img/circle200x200.png**.

We willen bij de verschillende controles (zoals de controle op een win-situatie) de naam van het afbeeldingsbestand welke getoond wordt door de **IMG-tag** vergelijken met de naam van een *afbeeldingsbestand* welke we willen vergelijken met het *afbeeldingsbestand* die getoond wordt.

Dus we willen eigenlijk een controle (vergelijking) uitvoeren die er als volgt uit ziet:

IMG-tag src-attribuut bevat de naam **img/cross200x200.png**

De vragen die bij ons dan naar boven komen zijn:

- "Hoe vergelijk ik de namen van deze twee afbeeldingsbestanden met elkaar?"
- "Kunnen we dat op dezelfde manier doen als bij de controle in de `cellClick` functie, waar we controleren of b.v. de tekst 'Start ronde' op de knop doet?"
- "Zo niet, hoe moet het dan?"

Op deze vragen gaan we eerst het antwoord zoeken. Pas daarna gaan we de functie programmeren.

WAT IS DE CELLCLICK FUNCTIE VOOR EEN FUNCTIE?

EventHandler

De **cellClick** functie is wat we noemen een *EventHandler*. Dit betekent niks anders dan dat het een functie is die gekoppeld is aan een gebeurtenis, zoals bijvoorbeeld een click op een element. Iedere functie die reageert op een gebeurtenis op de pagina noemen we dus een EventHandler.

De browser **verzameld allerlei gegevens over een gebeurtenis** in een object en geeft deze door aan de functie die op de gebeurtenis moet reageren (EventHandler dus).

De functie krijgt de gegevens van de browser binnen via een **parameter**.

Wat is een parameter?

Een parameter is te vergelijken met een variabele, maar dan een variabele die alleen geldt voor een bepaalde functie. Dus de parameter leeft alleen zolang de betreffende functie z'n werk doet.

Een parameter wordt bij de declaratie van een functie (daar in de code waar de functie geschreven is) tussen de haakjes van de functie geplaatst en een naam gegeven (net als bij variabelen). Een functie kan 1 of meerdere parameters hebben.

De naam van een parameter mogen we zelf bepalen, maar 'Best Practice' is dat de naam van een parameter altijd duidelijk maakt wat voor gegevens erin zitten.

Voorbeelden

Voorbeeld 1

```
function voorbeeld1( parameter )  
{  
  
}
```

*Figuur 2 - Declaratie van de functie met 1 parameter
Gebruik van de functie met 1 parameter*

```
voorbeeld1( 'Dit is de content van de parameter' );
```

Figuur 3 - Aanroepen van voorbeeldfunctie 1

Voorbeeld 2

```
function voorbeeld2( parameter1 , parameter2 )  
{  
  
}
```

Figuur 4 - Declaratie van de functie met 2 parameters

```
voorbeeld2( 'Content van de 1e parameter', 2 );
```

Figuur 5 - Aanroepen van voorbeeldfunctie 2

Parameter van een EventHandler

Een EventHandler heeft altijd 1 parameter. Deze parameter wordt automatisch door de browser gevuld met gegevens over de gebeurtenis. Maar als we de parameter niet benoemen in de declaratie van de EventHandler functie dan ontvangt onze functie ook niks. We moeten de parameter dus wel benoemen.

Voorbeelden

Voorbeeld 1

```
/*  
 * Deze functie handelt iedere klik op een speelveld cel af.  
 */  
function cellClick()  
{  
    
}
```



Figuur 6 - Geen parameter benoemd

In het bovenstaande voorbeeld hebben we geen parameter voor de EventHandler **cellClick** benoemd. Dit betekent dan ook dat de browser geen gegevens over de gebeurtenis kan doorgeven aan de functie.

Voorbeeld 2

```
/*  
 * Deze functie handelt iedere klik op een speelveld cel af.  
 */  
function cellClick(event_element)  
{  
    
}
```



Figuur 7 - Zelfde functie, maar nu wel een parameter benoemd

In bovenstaand voorbeeld hebben we wel een parameter benoemd. We hebben het de naam **event_element** gegeven.

Dit betekent dat de browser nu wel de gegevens van de gebeurtenis kan doorgeven aan de functie.

De **reden** waarom we de naam **event_element** hebben gekozen is overduidelijk. De naam geeft nu duidelijk aan dat het gaat om event gegevens van een element op de pagina.

Parameter = Argument

Een parameter van een functie wordt ook wel argument (Engelse uitspraak) genoemd.

WELKE GEGEVENS VINDEN WE IN DE PARAMETER VAN CELLCLICK?

Als we alles tot nu toe goed hebben gedaan en geprogrammeerd dan kunnen we nu testen welke gegevens in de parameter **event_element** van de **cellClick functie** zitten.

We gaan daarom **de cellClick functie** voorzien van 1 regel code om de gegevens in de console van de inspector te tonen en lopen daardoorheen.

Testen

Tijdelijk plaatsen we de volgende regel in de **cellClick** functie:

```
/*
 * Deze functie handelt iedere klik op een speelveld cel af.
 */
function cellClick(event_element)
{
  console.log(event_element);
}
```

Figuur 8 - De opdracht `console.log` om te testen

De opdracht **console.log** is een standaard functie in JavaScript en een mooie functie om in de console van de inspector te controleren of onze code doet wat het moet doen. Hierin is **console** een object en **.log** een functie (method) in dat object.

Er zijn meerdere mogelijkheden met het **console object**, dus niet alleen **.log** als functie. Hieronder vind je een link naar een pagina die je laat zien wat voor mogelijkheden je nog meer hebt.

[LINK: W3Schools - Console opdrachten](#)

1. Nu laden we ons spel in de browser en openen de inspector.
2. In de inspector openen we de console
3. We klikken op de button van ons spel
Dit is nodig, omdat we dan pas de cellen klikbaar maken.
4. We klikken nu op een willekeurige cel in ons spel en kijken wat er in de console van de inspector verschijnt.



Figuur 9 - In de console van de inspector zien we nu de inhoud van `event_element`

Welke property in `event_element` kunnen we gebruiken?

Na onderzoek blijkt dat het object, welke door de browser is samengesteld en als parameter aan onze functie is doorgegeven, een **target** property bevat welke wijst naar het element waarop geklikt is. Na het uitproberen in de console van de inspector kwamen we erachter dat target in feite een **IMG-object** (oftewel een **IMG-tag** in het speelveld) is en dus ook beschikt over de property **.src**. Daarmee kunnen we het afbeeldingsbestand van de **IMG-tag** wijzigen naar een ander afbeeldingsbestand.

Conclusie

In **event_element** gaan we de property **target** gebruiken om toegang te krijgen tot het element waarop geklikt is.

SRC-ATTRIBUUT IMG-TAG VERGELIJKEN MET TEKST

Inleiding

In de HTML-code, maar ook in onze JavaScript-code, hebben we de **src**-attribuut gevuld op de volgende manier:

```

```

Figuur 10 - Image-tag

Dus met de tekst: **"img/empty.jpg"**

Dezelfde tekst gebruiken we ook in onze JavaScript-code. We beperken ons altijd tot de relatieve **map img** (relatief t.o.v. de root van onze website) gevolgd door: **/<bestandsnaam>**.

Maar na wat uitproberen in de console van de inspector komen we erachter dat de browser de **src**-attribuut op de achtergrond heeft aangevuld op de volgende manier:

```
spellcheck: true  
src: "http://127.0.0.1:5500/student/img/empty.jpg"  
srcset: ""  
▶ style: CSSStyleDeclaration {accentColor: '', additiveSym
```

Figuur 11 - Browser vult op de achtergrond de link in de SRC-attribuut aan

We zien hier heel duidelijk dat de browser de src-attribuut dus volledig maakt met de volledige URL naar het afbeeldingsbestand.

Wat betekent dit voor ons?

Dit betekent dat we in onze JavaScript-code niet zomaar de tekst van de src-attribuut kunnen vergelijken met b.v. de tekst **"img/empty.jpg"**. Dit komt dus nooit overeen en levert dus nooit een WAAR op. En dit terwijl we duidelijk zien aan de afbeelding hierboven dat het toch om hetzelfde afbeeldingsbestand gaat.

We kunnen er van tevoren niet van uitgaan dat we weten welke URL hier komt te staan. Op dit moment is het `http://127.0.0.1/student/`, maar als we ons spel straks online zetten onder welke URL doen we het dan? En eigenlijk willen we onze code zo onafhankelijk maken van welke domeinnaam dan ook.

Dit levert ons de vraag op:

"Hoe kunnen we de tekst van een SRC-attribuut vergelijken met tekst uit onze code?"

ZOEKEN NAAR EEN ANTWOORD OP ONZE VRAAG

Inleiding

Op de vraag “Hoe kunnen we de tekst van een SRC-attribuut vergelijken met tekst uit onze code?” moeten we dus nu een antwoord vinden. Daarvoor gebruiken we weer het internet.

Wat is een betere zoekvraag voor het internet?

Hoe controleren we of een **SRC**-attribuut tekst zoals “*img/empty.jpg*” bevat. En **bevat** is daarbij een belangrijke Keyword.

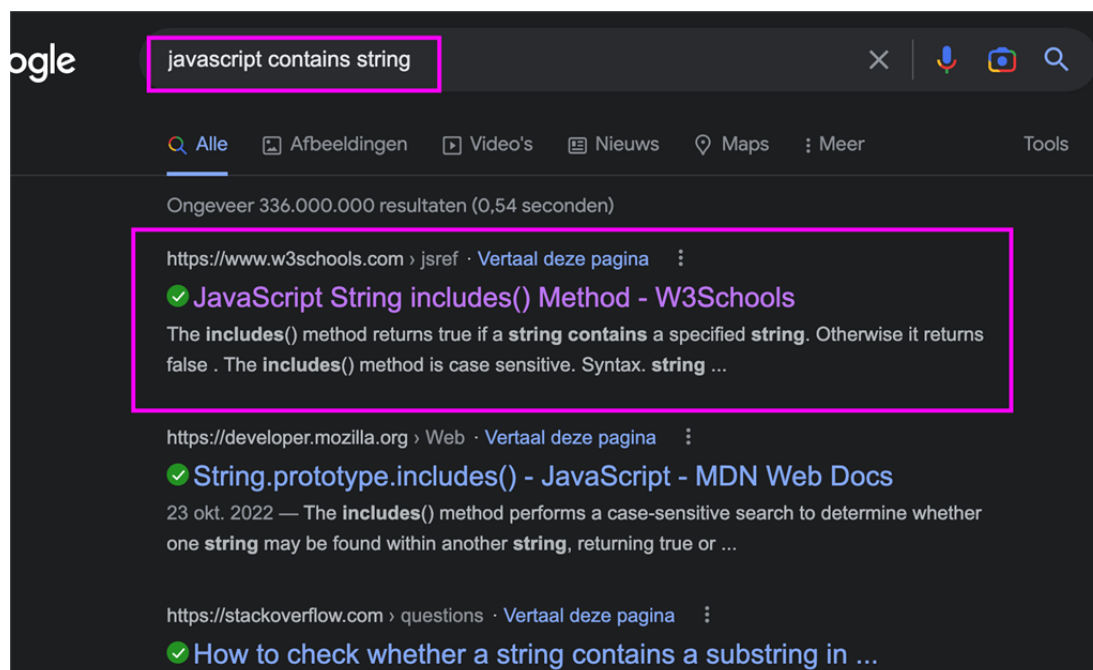
We zoeken altijd in het Engels, dus tikken we in de zoekmachine de volgende zoekterm in:

`javascript contains string`

Contains is het Engelse woord voor bevat

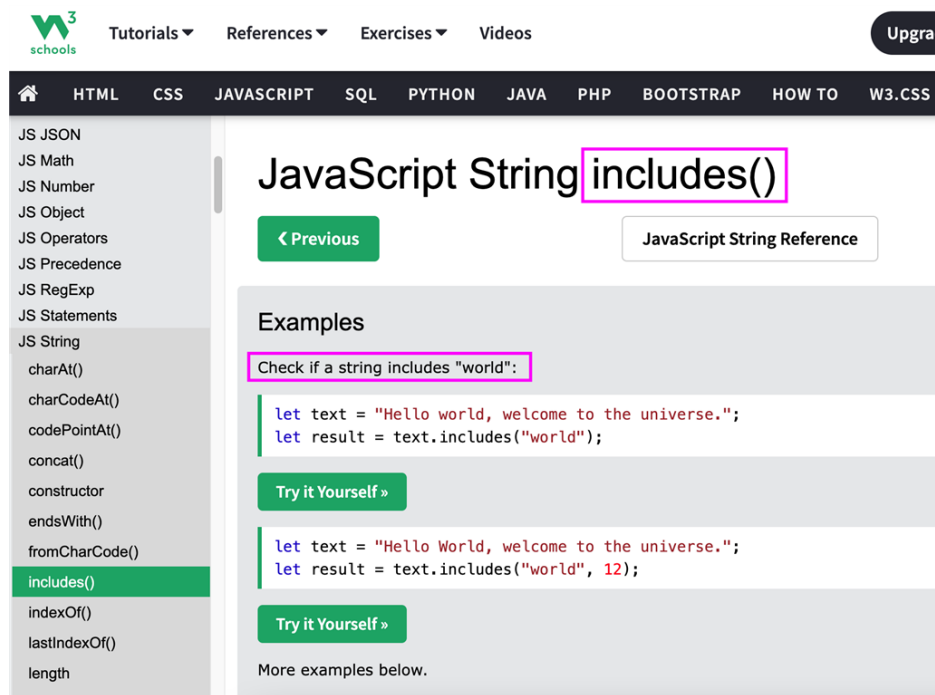
string omdat we te maken hebben met een gegeven van het type string.

Wat vinden we dan?



Figuur 12 - Resultaat van de zoekterm

Na het klikken op de eerste link die ons geboden wordt komen we op de volgende pagina terecht:



Figuur 13 - Een van de gevonden oplossingen

[LINK: W3Schools - String Includes method](#)

Aangezien de **SRC**-attribuut ook een gegeven bevat van het **type string** moeten we deze method (functie) ook kunnen gebruiken. Maar dit gaan we eerst in de console van de inspector testen.

Tijdens de voorbereidingen hebben we al in kaart gebracht in een schets wat de functie aan taken moet uitvoeren na een klik op een cel in het speelveld. Dit pakken we er nog even weer bij.



Rood omkaderd zien we in de *figuur 14* hierboven wat de functie moet doen. Dit moeten we dus omzetten naar programma-code in onze functie.

De taken

Zoals uit de afbeelding in de inleiding is af te leiden moet onze functie dus de volgende taken uitvoeren (en niet persé in de volgorde waarin ze staan):

- **Controleren of de cel waarop geklikt is leeg is**

Want dan kan pas het symbool van een speler in de cel geplaatst worden. Dit is eigenlijk niet echt nodig, want als een cel gevuld wordt met het symbool van de speler dan wordt dezelfde cel onklikbaar gemaakt. Dus kan er nooit op een al gevulde cel geklikt worden. Maar voor de zekerheid gaan we deze taak wel uitvoeren.

- **Cel is leeg:**

- We **plaatsen het symbool** van de huidige speler in de cel
 - We maken de **cel onklikbaar**
 - We **wisselen de beurt** naar de andere speler
 - We **tonen de beurt** van de andere speler in de UI
 - **Controleren van de win-situaties**

We controleren of er een speler is die na deze zet gewonnen heeft.

- **JA**

- **Punten toekennen** aan winnende speler
 - **Nieuwe score tonen** in de UI
 - **Ronde resetten/stoppen**

- **NEE**

- **Controleren op gelijk spel**

We controleren of er nog minstens 1 cel gevuld is met **'img/empty.jpg'**.

- **JA**, ronde kan gewoon doorgaan en speler actie gewoon uitvoeren.
 - **NEE**, alle cellen zijn bezet. Dan hebben we nu een gelijkspel en krijgen beide spelers punten en laten we van beide spelers de nieuwe score zien in de UI.

- **Cel is NIET leeg:** We doen niets

Taak: Speler actie uitvoeren

Resultaat in code

```
148  /*
149  * Deze functie handelt iedere klik op een speelveld cel af.
150  */
151  function cellClick(event_element)
152  {
153      if (event_element.target.src.includes('img/empty.jpg')) {
154          // Cel is leeg, dus plaatsen we het symbool van de huidige speler
155          if (current_player == 1)
156              event_element.target.src = 'img/cross200x200.png';
157          else
158              event_element.target.src = 'img/circle200x200.png';
159
160          // Cel onklikbaar maken
161          event_element.target.removeEventListener('click', cellClick);
162
163          // Wisselen van beurt
164          current_player = (current_player == 1 ? 2 : 1);
165
166          // Tonen van de beurt
167          element_turn_number.innerHTML = current_player;
168          element_turn_image.src = (current_player == 1 ? 'img/cross200x200.png' : 'img/circle200x200.png');
169
170          // TODO: Hieronder komt de code voor het controleren van de win-situaties
171      }
172  }
173
174  // Als de cel niet leeg is dan doen we niets, dus hoeven we hier geen code
175  // te plaatsen
176  }
```

Figuur 15 - Speler actie uitvoeren

Toelichting op de code

Regel 153

We passen de eerder gevonden methode toe om te controleren of de cel ook echt leeg is. Als deze leeg is staat het afbeeldingsbestand `'img/empty.jpg'` in de **SRC**-attribuut van de **IMG-tag** vermeld.

Regel 155 t/m 158

We controleren eerst welke speler aan de beurt is. We houden dit namelijk bij in de hulpvariabele `current_player`. Als dit 1 is dan plaatsen we een kruis als symbool in de cel en anders (dan zal het wel 2 zijn) plaatsen een cirkel als symbool in de cel.

Regel 161

Nu is het tijd om de cel zelf onklikbaar te maken, zodat spelers er niet nog een keer op kunnen klikken.

Regel 164

We wisselen nu de beurt. Je ziet dat we dat weer doen met een ternary operator. Dit hadden we ook bij regel 155 kunnen doen overigens.

Regel 167

We laten het nummer van de speler die nu aan de beurt is zien in de UI.

Regel 168

We laten nu ook het symbool zien van de speler die aan de beurt is.

Regel 170

Hier gaan we in een volgende stap onze controles uitvoeren die moeten bepalen of er een speler is die de ronde gewonnen heeft. (Zie eerdere voorbereidingen)

Win situaties controleren

Inleiding

In een voorbereiding hebben we met pseudo-code de controles beschreven. Dit gaan we nu omzetten in kleine stappen naar JavaScript-code.

Van pseudo-code naar JavaScript

In de pseudo-code maakten we gebruik van verschillende woorden. In de tabel hieronder komen ze terug, maar dan in de tweede kolom de JavaScript vertaling ervan:

Vertaling van pseudo-code naar JavaScript	
PSEUDO-CODE	JAVASCRIPT VERTALING
<i>als</i>	if()
EN	&&
OF	
ANDERS	else

Eerste pseudo-code controle omzetten naar JavaScript

De eerste die we hadden samengesteld betrof de controle of een speler met het symbool Cirkel op de eerste rij heeft gewonnen. De pseudo-code ziet er als volgt uit:

als **cel1** bevat Cirkel **EN** **cel2** bevat Cirkel **EN** **cel3** bevat Cirkel

Als voorbeeld gaan we deze nu omzetten naar JavaScript-code.

Opmerkingen vooraf

De afbeeldingen in de cellen staan in de globale variabele **elements_cells**.

Deze variabele is een **ARRAY** van cellen. In de pseudo-code staat b.v. **cel1** maar in de variabele **element_cells** is dit de cel met **index 0**. In array's beginnen indices (meervoud index) altijd vanaf 0. Dus met **cel1** bedoelen we dus **element_cells[0]**.

In JavaScript kennen we het keyword **EN** niet. In JavaScript gebruiken we daarvoor **&&**.

We hadden al in onze onderzoekjes ontdekt dat we het woord **bevat** kunnen vervangen door de method **includes**.

De aanduiding **Cirkel** in de pseudo-code moeten we uiteraard vertalen naar een echte afbeelding, en dat wordt dan **'img/circle200x200.png'**.

Resultaat

PSEUDO CODE:

als **cel1** bevat Cirkel **EN** **cel2** bevat Cirkel **EN** **cel3** bevat Cirkel

JAVASCRIPT CODE: *(Voor de leesbaarheid verdeeld over meerdere regels)*

```
if (  
    element_cells[0].src.includes('img/circle200x200.png') &&  
    element_cells[1].src.includes('img/circle200x200.png') &&  
    element_cells[2].src.includes('img/circle200x200.png')  
)
```


Vertaling van de complete pseudo-code

Nadat we alle pseudo-code regels op deze manier hebben vertaald gaan we de laatste stap uitvoeren net als bij de laatste stap van de pseudo-code, en dat is het samenvoegen tot één geheel.

Pseudo-code (compleet voor 1 speler)

Daarbij gaan we eerst de rijen controleren, daarna de kolommen en tot slot de diagonalen en dat allemaal in 1 ALS-statement.

```
als (  
  (  
    (cel1 bevat Cirkel EN cel2 bevat Cirkel EN cel3 bevat Cirkel)      rij 1  
    OF (cel4 bevat Cirkel EN cel5 bevat Cirkel EN cel6 bevat Cirkel)   rij 2  
    OF (cel7 bevat Cirkel EN cel8 bevat Cirkel EN cel9 bevat Cirkel)   rij 3  
  ) OF (  
    (cel1 bevat Cirkel EN cel4 bevat Cirkel EN cel7 bevat Cirkel)      kolom 1  
    OF (cel2 bevat Cirkel EN cel5 bevat Cirkel EN cel8 bevat Cirkel)   kolom 2  
    OF (cel3 bevat Cirkel EN cel6 bevat Cirkel EN cel9 bevat Cirkel)   kolom 3  
  ) OF (  
    (cel1 bevat Cirkel EN cel5 bevat Cirkel EN cel9 bevat Cirkel)      Diagonaal 1  
    OF (cel3 bevat Cirkel EN cel5 bevat Cirkel EN cel6 bevat Cirkel)   Diagonaal 2  
  )  
)
```

JavaScript vertaling (compleet voor 1 speler)

(Voor de leesbaarheid verdeeld over meerdere regels)

```
if (  
  (  
    Rij 1 ↓  
    element_cells[0].src.includes('img/circle200x200.png')  
    && element_cells[1].src.includes('img/circle200x200.png')  
    && element_cells[2].src.includes('img/circle200x200.png')  
  ) || (  
    Rij 2 ↓  
    element_cells[3].src.includes('img/circle200x200.png')  
    && element_cells[4].src.includes('img/circle200x200.png')  
    && element_cells[5].src.includes('img/circle200x200.png')  
  ) || (  
    Rij 3 ↓  
    element_cells[6].src.includes('img/circle200x200.png')  
    && element_cells[7].src.includes('img/circle200x200.png')  
    && element_cells[8].src.includes('img/circle200x200.png')  
  )  
)  
)
```

LET OP!!!!

Let vooral op de haakjes in zowel de pseudo-code als de JavaScript vertaling. De haakjes zijn absoluut noodzakelijk, want daarmee groeperen we delen van de controle.

Eindresultaat

Hieronder vind je de code die we uiteindelijk moeten opleveren:

```
170 // Controleren van de win-situaties
171 if ( // SPELER 1 (cross)
172     ( // Rijen
173         ( // Rij 1
174             element_cells[0].src.includes('img/cross200x200.png') 66
175             element_cells[1].src.includes('img/cross200x200.png') 66
176             element_cells[2].src.includes('img/cross200x200.png')
177         ) || ( // Rij 2
178             element_cells[3].src.includes('img/cross200x200.png') 66
179             element_cells[4].src.includes('img/cross200x200.png') 66
180             element_cells[5].src.includes('img/cross200x200.png')
181         ) || ( // Rij 3
182             element_cells[6].src.includes('img/cross200x200.png') 66
183             element_cells[7].src.includes('img/cross200x200.png') 66
184             element_cells[8].src.includes('img/cross200x200.png')
185         )
186     ) || ( // Kolommen
187         ( // Kolom 1
188             element_cells[0].src.includes('img/cross200x200.png') 66
189             element_cells[3].src.includes('img/cross200x200.png') 66
190             element_cells[6].src.includes('img/cross200x200.png')
191         ) || ( // Kolom 2
192             element_cells[1].src.includes('img/cross200x200.png') 66
193             element_cells[4].src.includes('img/cross200x200.png') 66
194             element_cells[7].src.includes('img/cross200x200.png')
195         ) || ( // Kolom 3
196             element_cells[2].src.includes('img/cross200x200.png') 66
197             element_cells[5].src.includes('img/cross200x200.png') 66
198             element_cells[8].src.includes('img/cross200x200.png')
199         )
200     ) || ( // Diagonalen
201         ( // Diagonaal 1
202             element_cells[0].src.includes('img/cross200x200.png') 66
203             element_cells[4].src.includes('img/cross200x200.png') 66
204             element_cells[7].src.includes('img/cross200x200.png')
205         ) || ( // Diagonaal 2
206             element_cells[2].src.includes('img/cross200x200.png') 66
207             element_cells[4].src.includes('img/cross200x200.png') 66
208             element_cells[6].src.includes('img/cross200x200.png')
209         )
210     )
211 ) {
212     // Hier voeren we de code uit wanneer speler 1 ergens heeft gewonnen
213 } // EINDE SPELER 1 WIN CONTROLE
214 else if ( // SPELER 2 (circle)
215     ( // Rijen
216         ( // Rij 1
217             element_cells[0].src.includes('img/cross200x200.png') 66
218             element_cells[1].src.includes('img/cross200x200.png') 66
219             element_cells[2].src.includes('img/cross200x200.png')
220         ) || ( // Rij 2
221             element_cells[3].src.includes('img/cross200x200.png') 66
222             element_cells[4].src.includes('img/cross200x200.png') 66
223             element_cells[5].src.includes('img/cross200x200.png')
224         ) || ( // Rij 3
225             element_cells[6].src.includes('img/cross200x200.png') 66
226             element_cells[7].src.includes('img/cross200x200.png') 66
227             element_cells[8].src.includes('img/cross200x200.png')
228         )
229     ) || ( // Kolommen
230         ( // Kolom 1
231             element_cells[0].src.includes('img/cross200x200.png') 66
232             element_cells[3].src.includes('img/cross200x200.png') 66
233             element_cells[6].src.includes('img/cross200x200.png')
234         ) || ( // Kolom 2
235             element_cells[1].src.includes('img/cross200x200.png') 66
236             element_cells[4].src.includes('img/cross200x200.png') 66
237             element_cells[7].src.includes('img/cross200x200.png')
238         ) || ( // Kolom 3
239             element_cells[2].src.includes('img/cross200x200.png') 66
240             element_cells[5].src.includes('img/cross200x200.png') 66
241             element_cells[8].src.includes('img/cross200x200.png')
242         )
243     ) || ( // Diagonalen
244         ( // Diagonaal 1
245             element_cells[0].src.includes('img/cross200x200.png') 66
246             element_cells[4].src.includes('img/cross200x200.png') 66
247             element_cells[7].src.includes('img/cross200x200.png')
248         ) || ( // Diagonaal 2
249             element_cells[2].src.includes('img/cross200x200.png') 66
250             element_cells[4].src.includes('img/cross200x200.png') 66
251             element_cells[6].src.includes('img/cross200x200.png')
252         )
253     )
254 ) {
255     // Hier voeren we de code uit als speler 2 ergens heeft gewonnen
256 } // EINDE SPELER 2 WIN CONTROLE
257 // TODO! Gelijk spel controle moet hier nog plaats vinden
258
259
260
```

Figuur 16 - Complete IF-statement

*We kunnen nu al constateren dat deze controle een enorm **GEDROCHT** is en hij is nog niet eens af :-)*

Als we nu al constateren dat alleen al de controle een **GEDROCHT** is dan weten we nu al dat we daar later mee aan de slag moeten om dit gedrocht te optimaliseren.

Programmeren van de acties

We gaan nu programmeren wat er moet gebeuren als b.v. speler 1 heeft gewonnen en wat er moet gebeuren wanneer speler 2 heeft gewonnen.

Speler 1

Voor speler 1 coderen we het volgende:

```
212 | {  
213 |     // SPELER 1 HEEFT GEWONNEN  
214 |     // STAP 1 - Punten toekennen aan speler 1  
215 |     score_player1 += 2;  
216 |     // STAP 2 - Score tonen  
217 |     element_score_player1.innerHTML = score_player1;  
218 |     // STAP 3 - Ronde resetten  
219 |     element_game_button.click();           // TRUCJE :-)  
220 |  
221 | } // EINDE SPELER 1 WIN CONTROLE
```

Figuur 17 - Acties wanneer speler 1 heeft gewonnen

Regel 215

We kennen speler 1 nu 2 punten toe. Want we hadden in de voorbereidingen besloten om een winnaar 2 punten toe te kennen, een verliezer 0 punten, en bij een gelijk spel krijgen beide spelers 1 punt.

We houden de score bij in de hulpvariabele `score_player1`.

Regel 217

Nu tonen we de nieuwe score in onze UI.

Regel 219

Dit is een trucje. De functie **buttonClick** bevat al code voor een reset van het spel. In de variabele **`element_game_button`** bevindt zich de button in het spel. Als we alles goed hebben geprogrammeerd staat de tekst '*Reset ronde*' tijdens het spelen op de button. Als we nu een click op de button kunnen simuleren wordt de code uitgevoerd om de ronde te resetten. En dit is precies wat met de code van deze regel doen.

Met **`.click()`** achter de variabele **`element_game_button`** simuleren we een click van een speler op de button.

Speler 2

```
262     ) {  
263         // SPELER 2 HEEFT GEWONNEN  
264         // STAP 1 - Punten toekennen aan speler 2  
265         score_player2 += 2;  
266         // STAP 2 - Score tonen  
267         element_score_player2.innerHTML = score_player2;  
268         // STAP 3 - Ronde resetten  
269         element_game_button.click();           // TRUCJE :-)  
270  
271     } // EINDE SPELER 2 WIN CONTROLE  
272     // TODO: Gelijk spel controle moet hier nog plaats vinden
```

Figuur 18 - Acties wanneer speler 2 heeft gewonnen

Toelichting op de code

De code in dit deel is niet veel anders, behalve dan dat we de punten nu toekennen aan speler 2.

Gelijkspel

We hebben nu ontdekt bij het programmeren van de controles op een winnaar dat we een enorm gedrocht krijgen in de if-statement. We willen dit voor de controle op een gelijk spel nu voorkomen.

Hoe pakken we het dan nu aan?

We moeten ons eerst de vraag stellen:

'Wanneer is er sprake van een gelijkspel?'

Het antwoord daarop is:

Wanneer er geen enkele cel meer leeg is.

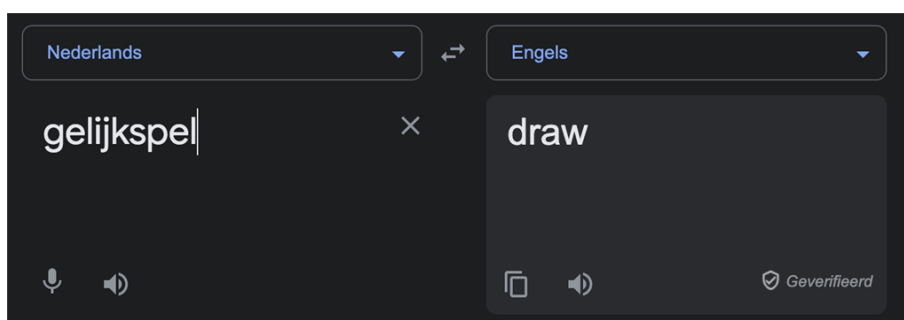
Dus in feite hoeven we alleen maar langs alle cellen te lopen en te kijken of er op z'n minst nog 1 cel leeg is. Zodra we dan een cel tegenkomen die leeg is kunnen we stellen dat er nog GEEN sprake is van een gelijk spel. We kunnen het ook omdraaien: als we o cellen tegenkomen die leeg zijn is er WEL sprake van gelijk spel.

Dit gaan we nu programmeren. Maar we gaan daarvoor nu wel een aparte functie maken die dit allemaal doet en met een **return-statement** de IF een WAAR of een ONWAAR geeft.

Hoe gaat de functie dan heten?

Dat is afhankelijk van wanneer de functie een WAAR terug gaat geven. Wij bedenken nu dat de functie een WAAR terug moet geven wanneer er WEL sprake is van een gelijk spel. We lopen daarbij langs alle cellen en besluiten dat, zodra we een lege cel tegenkomen we een ONWAAR teruggeven. Als we langs alle cellen zijn gelopen en dus geen lege cel zijn tegengekomen is de standaardwaarde die de functie teruggeeft een WAAR is.

De functie noemen we dan **isDraw**



Figuur 19 - Google vertaling van gelijkspel

Zo zou de functie er dan uit kunnen zien

```
299  /*
300  * Dit is een helper function
301  * Deze functie gebruiken we om te controleren of
302  * er sprake is van een gelijk spel
303  *
304  * Return value:
305  * TRUE      Er is sprake van een gelijkspel
306  * FALSE     Geen gelijk spel
307  */
308  function isDraw()
309  {
310      for (let cell of element_cells) { // Lus om langs alle cellen te lopen
311          if (cell.src.includes('img/empty.jpg'))
312              return false; // We zijn een lege cel tegengekomen
313      }
314
315      return true; // Als we hier komen zijn we geen lege cel tegengekomen
316  }
```

Figuur 20 - Zo zou de isDraw functie eruit kunnen zien

Toelichting

Regel 310

Dit is een lus functie van JavaScript welke ons in staat stelt langs de array element_cells te lopen, waarbij iedere keer we de lus doorlopen een cel uit de array in de tijdelijke variabele cell geplaatst wordt. Deze tijdelijke variabele kunnen we dan in de lus gebruiken voor de controle op regel 311.

[LINK: W3Schools - for....of.... lus](#)

Regel 311 en 312

Op regel 311 controleren of de cell, die we in de lus dan te pakken hebben, in de SRC-attribuut de tekst 'img/empty.jpg' heeft staan. Als dat zo is dan geven we met de statement op regel 312 een false terug.

Regel 315

Op deze regel komen we pas wanneer we in de lus geen enkele lege cel zijn tegengekomen. Eigenlijk vertellen we met deze regel dat de standaard waarde die deze functie teruggeeft een WAAR is, oftewel we gaan er vanuit dat er altijd een gelijkspel is tenzij in de lus het tegendeel bewezen wordt.

Hoe past dit in de gehele controle?

```
250         ) || ( // Diagonalen
251         ( // Diagonaal 1
252             element_cells[0].src.includes('img/cross200x200.png') &&
253             element_cells[4].src.includes('img/cross200x200.png') &&
254             element_cells[7].src.includes('img/cross200x200.png')
255         ) || ( // Diagonaal 2
256             element_cells[2].src.includes('img/cross200x200.png') &&
257             element_cells[4].src.includes('img/cross200x200.png') &&
258             element_cells[6].src.includes('img/cross200x200.png')
259         )
260     )
261 } {
262     // SPELER 2 HEEFT GEWONNEN
263     // STAP 1 - Punten toekennen aan speler 2
264     score_player2 += 2;
265     // STAP 2 - Score tonen
266     element_score_player2.innerHTML = score_player2;
267     // STAP 3 - Ronde resetten
268     element_game_button.click(); // TRUCJE :-)
269 } // EINDE SPELER 2 WIN CONTROLE
270
271 else if (
272     isDraw() // Controleer op gelijkspel
273 ) {
274     // GELIJKSPEL
275     score_player1 += 1;
276     score_player2 += 1;
277     element_score_player1.innerHTML = score_player1;
278     element_score_player2.innerHTML = score_player2;
279
280     element_game_button.click();
281 }
282
```

Figuur 21 - Functie isDraw in de IF-statement

Aan de hand van bovenstaande code zien we ook gelijk dat we bij een gelijkspel beide spelers 1 punt geven, de nieuwe scores in onze UI tonen en tot slot een reset van de ronde activeren door een click op de button te simuleren.